

Unified single-head inference: evaluation and recommendation

Removing the separate seen/unseen classifier heads in favour of a two-stage shortlist-then-decide pipeline. Questions answered, changes made, measured results.

The proposal was to drop the two classifier heads and run every evaluation image through one pipeline: stage 1 compares the image against vision prototypes and keeps the top-k most similar seen classes; stage 2 makes the final call by comparing the image against text prompts for those k classes plus all unseen classes. I built it, measured it four ways, and the answer is that it costs us accuracy rather than gaining any. Details below.

Bottom line. The unified pipeline scores best on our internal holdout (57.02 vs 50.71) and worst on the eval set, projecting **49.35% at best against the 51.44% we have already banked**. The holdout is actively misleading on this axis: it ranks our two live configurations in the wrong order by five points. No submission slot was spent.

Questions answered

1. Are the prototypes pre-extracted from all training images?

Yes. Each class prototype is the mean of the embeddings of every training image belonging to that class, computed once from cached embedding files rather than at inference time. Two things worth knowing that the question implies but the code does not: the prototype pool is **64,259** images, not the full 99,979, because we intersect across the seven encoder embedding files and only images present in all of them are usable; and the seen-class score is not the prototype alone. It is a sum of three terms:

$$\text{score} = \text{prototype_cosine} + 2.0 \times \text{cmax} + 4.0 \times \text{taxon_text_cosine}$$

where **cmax** is the highest cosine against any single training image of that class. That distinction matters for the second question.

2. Should cosine scores be re-weighted by training count instead of using the seen/unseen split explicitly?

The instinct is right and the effect is larger than one would guess. Training counts are extremely skewed: minimum 2, **median 2**, maximum 281. Half the seen classes have exactly two images, so this is not a rounding concern.

The cleanest place to apply it is the **cmax** term specifically, rather than the whole seen block. A maximum over n samples grows with n by construction, so a 281-image class gets 281 draws at the maximum while a 2-image class gets two. That is an upward bias for head classes independent of any true similarity, and it is exactly the kind of unfairness the re-weighting is meant to correct. The mean-prototype term is a different story, since its reliability depends on within-class spread and the sign is not obvious in advance.

A fixed top-2 mean per class neutralises the max-of-n bias directly and is feasible for every class given the minimum count of 2. A helper for this already exists in the shared research utilities. This was deferred as requested and is not part of the results below.

3. On the rules

The change was framed as fitting new rules. I re-fetched the live competition terms on 8 August and compared them against our cached copy from 23 July: the relevant clause is **unchanged, word for word**. There is no new rule here.

More importantly, the current pipeline does not use split information. The clause prohibits using the provided split files to identify whether an evaluation image is seen or unseen, or to restrict the candidate classes. Our gate is computed from the image and text embeddings alone, and the seen/unseen class partition is derived from the training labels file, never from the split files. The rules explicitly permit "a novelty/gate signal computed from the image/text itself".

So a unified pipeline is an architecture preference, not a compliance fix. That distinction is worth settling before we weigh the cost, because if it were a compliance fix we would have to accept the cost regardless.

What was built

One evaluation harness comparing four scoring architectures on identical inputs, identical class prototypes and identical text stacks, so the only variable is the pipeline shape. Stage 1 is the same vision-prototype comparison in every case; the arms differ only in what stage 2 scores with.

Arm	Stage-2 score function	Heads
A	Current production. A gate routes each image to a seen head (prototype argmax) or an unseen head (text plus external image prototypes).	Two
B	The proposal as specified: image-to-text similarity only, over the top-k seen classes plus all unseen classes.	One
C	B plus the external image-prototype legs extended across the full class space, so seen and unseen candidates receive the same evidence types.	One
D	C plus one uniform reference-photo leg: same encoder and same statistic on both sides, using a class's training images where it has them and external photos where it does not.	One

Arm D deserves a note: it is not in the original proposal. Arms B and C both leave one side of the candidate pool with evidence the other side cannot have, which is the same asymmetry the proposal set out to remove. D is the version that genuinely applies one rule to every class, and it is the strongest unified design the current artifacts support.

Two measurement surfaces were used. The internal holdout, where the rarest 20% of seen classes are withheld and treated as novel, gives labelled accuracy. The real 35,665-image evaluation batch has no labels, so it gives the routing diagnostic instead: what fraction of each folder's images land on a seen versus an unseen class. Folder membership is used for counting only and never enters any prediction.

Image and flipped image was already in place before this work. Horizontal-flip test-time augmentation is the default in every extraction path, alongside an aspect-corrected second view added to match the elongated framing of the evaluation images.

Results

Holdout accuracy, weighted overall

Arm	Seen	Unseen	Overall
A, production gate at $f = 0.60$	69.37	26.62	50.71
A, production gate at $f = 0.72$	81.04	23.21	55.80
B, proposal as specified ($k = 5$)	78.77	28.95	57.02
C, full-space image legs ($k = 5$)	66.87	41.54	55.82
D, uniform reference-photo leg ($k = 5$)	67.01	43.74	56.86

On this evidence the proposal wins by 1.2 points and we should ship it. That reading is wrong, and the next table is why.

The holdout does not survive its own calibration check

We have real scores for two configurations of the current pipeline, which lets us ask what the holdout says about a comparison whose answer we already know.

Configuration	Holdout says	Reality says
Production at $f = 0.60$	50.71	51.44
Production at $f = 0.72$	55.80	50.84
Verdict	$f = 0.72$ wins by 5.09	$f = 0.60$ wins by 0.60

The holdout orders the two configurations backwards, and by five points. The reason is structural: the holdout has no distribution shift, its stand-in novel classes are rare seen classes rather than genuinely unseen ones, and it therefore systematically rewards whichever pipeline commits harder to the seen route. Every unified arm does exactly that. A proxy that fails this badly on the one axis where we can check it cannot settle the question, and this is not the first time it has produced a result that reversed on submission.

Projection from the evaluation batch

The usable signal is routing behaviour on the real images. The production configuration reproduces our banked submission **exactly, all 35,665 predictions identical**, which means the conditional accuracies calibrated from it are measured rather than assumed: 88.0% on seen images kept on the seen route, 26.0% on unseen images caught by the unseen route. Applying those to each arm's routing gives an upper bound, because it credits every arm with the production head's accuracy.

Arm	Test imgs to seen class	Unseen imgs to unseen class	Projected overall	vs banked 51.44
A, production (banked)	86.7%	74.5%	51.44	anchor
B, proposal as specified	82.7%	73.4%	49.35	-2.10
C, full-space image legs	73.4%	84.4%	45.97	-5.47
D, uniform reference-photo leg	77.6%	74.3%	46.88	-4.56

These are ceilings, and the method is known to be generous: applied to the $f = 0.72$ configuration it over-predicts the real score by 1.23 points. Even granting every unified arm that much slack, they land between

48 and 50.5, all below what we already have. The unified arms also decide seen classes without the prototype ensemble, so their true conditional accuracy is lower than the figure credited to them here.

Why it loses

The two heads are not redundant machinery wrapped around one model. They carry different evidence, and a single score function cannot use both.

- The seen route runs a three-encoder prototype ensemble with a taxonomy-text anchor. Nothing equivalent exists on the unseen side, because unseen classes have no training images by definition.
- The unseen route leans on external photo prototypes. Those cover only 966 of 5,795 seen classes, so they cannot be applied evenly across the candidate pool as things stand.
- Any single score function therefore either drops the ensemble, which costs 4.0 points of seen routing in arm B, or lets the lopsided image evidence swamp the seen side, which costs 13.3 points in arm C.

Arm D closes the asymmetry as far as the current artifacts allow and matches production's unseen routing to within 0.2 points, 74.3 against 74.5. It still gives up **9.1 points** of seen routing. That gap is structural rather than a tuning shortfall: closing it requires re-embedding the external photo bank under the other two encoders so the unseen side gets the same ensemble. Those embeddings do not exist and represent new GPU work.

The shortlist size is not a lever either. Moving k from 5 to the full seen space changes holdout overall by 0.28, from 57.02 to 56.74. Stage 1 is contributing almost nothing; the unified pipeline is close to a plain argmax over the whole class space.

Two implementation traps

Both of these are live in the earlier two-stage builder and are worth fixing there regardless of what we decide about the architecture, since anyone rebuilding it will hit them.

Masking absent classes to a large negative value deletes them. Classes with no external prototype are masked to $-1e4$. Inside a dedicated unseen head that is harmless, since such a class was unreachable there anyway. In a unified score it removes the class from the entire candidate space, so a confident text match can never win. Absent evidence has to be neutral, not negative.

A normalised leg added to only one side of the partition decides the outcome by itself. The normalisation contributes a large constant offset per row. If that offset lands on the unseen columns and not the seen ones, the comparison is settled before any similarity is considered. In testing this produced a configuration that sent 99.7% of test images to seen classes and used a total of 94 unseen classes out of 11,598. That is a bug signature, not a result about unified inference, and it should not be quoted as one.

Recommendation

Keep the current two-head pipeline. It is compliant as it stands, and the unified alternative costs somewhere between two and five points against a banked result. No submission slot was spent on this and none should be; the previously staged two-stage packages belong to the same family as arm B and should not be submitted either.

If we want to revisit it, two things would have to change first:

- The external photo bank re-embedded under the remaining two encoders, so the unseen side of a unified score carries the same ensemble the seen side does. This is the only path that closes arm D's 9.1-point routing gap.
- A better proxy. The current holdout cannot rank routing architectures, and until something replaces it every change of this class has to be argued from routing behaviour and conditional accuracy rather than from proxy accuracy.

The count re-weighting question is separate from all of this and remains open on its own merits. It applies to the seen-class scoring inside whichever architecture we keep, and the max-of-n bias in the cmax term is a real and correctable effect. Worth testing next.

Harness and result files: `research/eval_unified_2stage.py`, with the plain invocation producing holdout accuracy and the `--real` flag producing the routing pass. Numbers land in `outputs/unified_2stage_holdout.json` and `outputs/unified_2stage_real_routing.json`. Findings recorded in the project handoff under 8 August.